

Preparando o conjunto de dados

Olá, pessoal!

Neste vídeo, iremos utilizar os primeiros passos do Tensorflow e do Keras. Keras era uma biblioteca que era utilizada como uma espécie de Front-end de outras bibliotecas de aprendizado de máquina, ou seja, o Keras era utilizado para facilitar a vida de quem queria utilizar bibliotecas mais complexas como era o caso do Tensorflow.

A partir da versão 1.4 do Tensorflow ele passou a incorporar o Keras dentro do seu pacote interno de bibliotecas.

Então, o Keras facilitou bastante a vida dos usuários do Tensorflow de modo que construir um modelo se tornou algo bem simples.

Neste vídeo, iremos utilizar o Tensorflow, o Keras e o NumPy como bibliotecas principais para manipular os dados que iremos utilizar durante esta aula.

O conjunto de dados utilizado nesta aula será um conjunto de dados de imagens de roupas.

Essas imagens de roupas são imagens de tamanho 28x28 e são cerca de 70 mil imagens que serão utilizadas no modelo.

Então, inicialmente, iremos importar o Tensorflow o Keras e o NumPy.

E isso será feito da seguinte maneira:

Na primeira célula, iremos escrever: `import tensorflow as tf`.

Estamos utilizando a abreviação "tf" para chamarmos o Tensorflow.

Em seguida, na linha seguinte, iremos ter: `from tensorflow import keras`.

Esse "keras" é justamente o Front-end que utilizaremos para o Tensorflow.

E, por fim, iremos importar o NumPy: `import numpy as np`.

Dessa maneira, teremos importado as três bibliotecas que iremos utilizar no decorrer desta aula.

Na próxima linha, iremos carregar o conjunto de dados.

E o conjunto de dados já está inserido na biblioteca Keras.

O nome deste conjunto de dados é "fashion_mnist".

O "mnist" original era um conjunto de dados constituído por dígitos escritos à mão.

O objetivo desse conjunto de dados, do "mnist" era tentar construir modelos que fossem capazes de reconhecer esses dígitos escritos à mão.

Para variar um pouco, foi criado o "fashion_mnist" que é uma variação, dez categorias de rótulos diferentes várias imagens delas e o objetivo é construir um modelo capaz de classificá-las corretamente.

Então, utilizaremos o seguinte código para carregar o "fashion_mnist": iremos carregar estes dados em uma variável chamada: `fashion_mnist = keras.datasets.fashion_mnist`, escrito da mesma maneira que foi dita anteriormente.

Com esse comando, poderemos carregar o conjunto de dados o "fashion_mnist" nesta variável "fashion_mnist".

O próximo passo é tentar separar, para fins de avaliação esse conjunto de dados em "train" e "test".

Esta variável "fashion_mnist" já vai fazer isso de uma maneira um pouco mais fácil para nós.

Então, na próxima linha, efetuaremos essa separação em "train" e "test":

Iremos escrever, na próxima linha, o seguinte: (train_images, train_labels), (test_images, test_labels). Dessa maneira, iremos armazenar as imagens de "train" e os seus rótulos no conjunto de dados. E aí, iremos armazenar as imagens de "test" e os seus rótulos de "test", respectivamente.

Após fechar parênteses, teremos: = fashion_mnist.load_data().

Essa função "load_data" já efetua a separação do conjunto de dados em um conjunto de "train" e "test", com os seus respectivos rótulos.

Iremos executar esses dois comandos o carregamento do "fashion_mnist" e a sua divisão em "train" e "test" e podemos conferir, na linha seguinte os tamanhos do conjunto de "train".

Então, utilizaremos o seguinte comando:

"train_images", ou seja, as imagens de "train" ".shape", que é justamente aquele atributo que nos diz qual é o tamanho do tensor armazenado.

Ao rodar esse comando, teremos o resultado que a variável "train_images", um tensor 3D onde nós temos 60 mil elementos de "train" cada um deles como uma matriz de 28x28.

Então, o resultado desse shape será: "(60000, 28, 28)".

Ou seja, são 60 mil elementos, cada elemento é uma matriz de 28x28.

A seguir, nós podemos verificar o tamanho dos rótulos de "train", também.

É esperado que os rótulos de "train" tenham o mesmo número de imagens do "train" ou seja, 60 mil.

Para verificar o tamanho do conjunto de rótulos de "train" iremos utilizar o seguinte comando: len(train_labels).

E iremos executar esse comando, onde iremos obter a resposta "60000" que é exatamente o mesmo número de elementos de "train".

Assim, podemos verificar quais são todos os rótulos que aparecem neste conjunto.

Para isso, iremos utilizar a biblioteca NumPye, que é uma biblioteca para tratar justamente com tensores, e utilizaremos da seguinte maneira: "np", que seria a abreviação de NumPye da maneira como nós importamos: np.unique(train_labels).

Ao executar este comando, nós iremos ter todos os labels que aparecem neste conjunto.

E os labels que nós temos são: 0, 1, 2, 3, 4, 5, 6, 7, 8 e 9, ou seja, de 0 a 9, nós temos 10 categorias diferentes.

Dessa maneira, conseguimos descobrir quais são os rótulos do "train" que nós temos e nós temos pelo menos um rótulo para cada uma dessas 10 classes.

Ou seja, os 60 mil rótulos estão distribuídos nessas 10 classes.

Podemos efetuar a mesma abordagem sobre o conjunto de "test" para verificar seus tamanhos.

Então, utilizando o comando: `test_images.shape`.

Executando este comando, iremos ter o resultado 10 mil imagens de 28x28.

Enquanto o conjunto de "train" possui 60 mil imagens, o conjunto de "test" tem 10 mil. Dessa maneira, conseguimos verificar quais são os tamanhos do conjunto de "test".

Podemos, também, utilizar o comando: `len(test_labels)` para verificar a quantidade de elementos, de rótulos de "test".

Temos que é igual a 10 mil, esses valores têm que bater tanto a quantidade de elementos de "test" quanto o número de rótulos de "test".

E, com uma pequena amostra desses rótulos de "test" nós podemos rodar apenas o comando "test_labels" para verificar quais são os rótulos de "test".

Ao executar, aqui, o rótulo da primeira imagem de "test" é "9" o rótulo da segunda imagem de "test" é "2" o da terceira é "1".

Nós temos vários rótulos de "test" e, por questão de limitação de espaço os 10 mil rótulos não são impressos ao executar este comando apenas os três primeiros e os três últimos.

Para finalizar essa preparação de dados nós podemos verificar que as imagens de "train" e as imagens de "test" são compostas por uma matriz de dimensões 28x28.

E cada um desses elementos da matriz é, exatamente, um pixel com valores entre 0 e 255 onde "0" é a ausência de cor do pixel e "255" é a intensidade máxima do pixel.

Em alguns modelos, tais como a regressão linear e as próprias redes neurais eles se beneficiam de atributos escalados.

Esses modelos utilizam uma técnica de gradiente para otimização, ou seja, de derivada.

Portanto, quando esses valores estão em escala pequena, como entre 0 e 1 esses modelos experimentam uma performance melhor.

Como todos os pixels estão com valores entre 0 e 255 nós podemos dividir as matrizes por 255.

Dessa maneira, o pixel com maior intensidade, que é o de 255 será igual a 1 o pixel de menor intensidade, que é 0, continuará sendo 0 e todos os valores intermediários estarão na escala entre 0 e 1.

Estamos utilizando um "MinMaxScaler" chamado explicitamente. Para isso, iremos utilizar: `train_images = train_images / 255.0`

e em seguida, iremos utilizar: `test_images = test / 255.0`

Dessa maneira, nós estamos forçando o resultado a ser um ponto flutuante e todos os valores, tanto das imagens de "train" quanto das de "test" estarão entre 0 e 1.

Com isso, teremos preparado o conjunto de dados para utilizar a rede neural.

Esta construção, esta preparação do conjunto de dados, é tão importante quanto a implementação do modelo, uma vez que dados que não estejam corretamente alinhados ou preparados, podem gerar modelos ruins ou modelos cuja avaliação não é confiável.

Então, nós temos um modelo que é capaz de ser executado de uma maneira bem confiável já que temos uma separação entre "train" e "test" para avaliação e estamos normalizando o conjunto de dados.

Encerramos este vídeo de construção e preparação do "dataset".

No próximo vídeo, falaremos um pouco sobre a construção do modelo de redes neurais, utilizando o Tensorflow.

Até lá!